



Programación 2
Trabajo Integrador
Food Store - Sistema de Gestión de
Pedidos de Comida

Institución: Universidad Tecnológica Nacional

Carrera: Tecnicatura Universitaria en Programación a Distancia

Materia: Programación 2

Estudiante: Luciano Emanuel Sosa

Fecha de entrega: 22/6/26

Indice

1. Marco teorico
2. Decisiones tecnicas y arquitectura
3. Modelo de dominio y relaciones
4. Funcionalidades implementadas
5. Pruebas de funcionamiento
6. Dificultades y soluciones
7. Bibliografia y webgrafia
8. Links de entrega

1. Marco teorico

Food Store fue desarrollado como una aplicación de consola en Java. El sistema representa categorías, productos, usuarios, pedidos y detalles de pedido mediante Programación Orientada a Objetos. La información se conserva en memoria durante la ejecución usando colecciones, de acuerdo con la consigna.

- Java 21: lenguaje utilizado para clases, enums, interfaces, excepciones y menú de consola.
- POO: encapsulamiento, constructores, herencia, toString y equals.
- Colecciones: listas para almacenar entidades activas e históricas.
- Excepciones propias: errores de negocio sin cierres inesperados.
- Baja lógica: eliminado = true en lugar de remover objetos.

2. Decisiones tecnicas y arquitectura

La solución se organizó en capas para mantener el código legible y evaluable. El menú solo interactúa con el usuario; los servicios concentran reglas de negocio y validaciones; las entidades representan el modelo de dominio.

Paquete	Responsabilidad
entities	Clases del dominio: BaseEntity, Categoria, Producto, Usuario, PerfilUsuario, Pedido y DetallePedido.
services	Casos de uso, validaciones, búsquedas y gestión de colecciones.
Main	Menú de consola y lectura validada con Scanner.
exception	Excepciones propias de negocio.
enums	Rol, Estado y FormaPago.
interfaces	Contrato Calculable para calcular totales.

Se agregó una relación 1:1 unidireccional entre Usuario y PerfilUsuario porque la pauta de corrección la menciona como regla excluyente. Cada usuario se crea con un perfil propio y no reutilizado.

3. Modelo de dominio y relaciones

- BaseEntity concentra id, eliminado y createdAt.
- Categoria tiene una relación 1:N con Producto.
- Producto mantiene una relación N:1 con Categoria.
- Usuario tiene una relación 1:N con Pedido y una relación 1:1 con PerfilUsuario.
- Pedido compone 1:N DetallePedido y usa Calculable.calcularTotal().
- DetallePedido referencia un Producto y guarda cantidad y subtotal.

4. Funcionalidades implementadas

Modulo	Operaciones
Categorías	Listar, crear, editar y eliminar logicamente. Impide eliminar si tiene productos activos.
Productos	Listar, crear, editar y eliminar logicamente. Valida precio, stock, categoria y disponibilidad.
Usuarios	Listar, crear, editar y eliminar logicamente. Valida mail obligatorio, formato basico y unicidad.
Pedidos	Listar, crear con detalles, actualizar estado/forma de pago y eliminar logicamente.

5. Pruebas de funcionamiento

El proyecto fue compilado con javac usando --release 21 y ejecutado desde consola. Se probaron listados de categorías, productos, usuarios y pedidos con datos iniciales.

```
=== SISTEMA DE PEDIDOS (FOOD STORE) ===
1. Categorías
2. Productos
3. Usuarios
4. Pedidos
0. Salir
Seleccione: 2

--- Productos ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: 1
Producto{id=1, nombre='Burger Clasica', precio=6500,00, stock=18, disponible=SI, categoria='Hamburguesas'}
Producto{id=2, nombre='Limonada', precio=1800,00, stock=29, disponible=SI, categoria='Bebidas'}
```

También se verifico que los pedidos calculen el total con la interfaz Calculable, que se descuenta stock al agregar detalles y que las bajas logicas oculten registros en listados posteriores.

6. Dificultades y soluciones

- La consigna principal no solicitaba base de datos, aunque el titulo menciona JDBC. Se resolvió respetando el alcance indicado: almacenamiento en memoria con colecciones.
- La pauta exige una relacion 1:1 real. Se incorporo Usuario -> PerfilUsuario para cubrir ese criterio sin alterar los CRUD principales.
- Para evitar estados inconsistentes, el pedido se confirma en memoria solo cuando tiene al menos un detalle valido.
- La baja logica se centralizo en servicios para que los listados muestren solo entidades no eliminadas.

7. Checklist de cumplimiento

La siguiente tabla resume como se cubrieron los puntos principales de la consigna y la pauta de corrección.

Requisito	Cumplimiento
POO y UML	Entidades separadas, herencia desde BaseEntity, encapsulamiento, constructores, equals y toString.
Colecciones	Servicios con listas en memoria para conservar registros activos e historicos.
Excepciones	NegocioException y derivadas para entidad no encontrada, duplicados, valores invalidos y stock insuficiente.
CRUD completo	Categorias, productos, usuarios y pedidos tienen listar, crear, editar y baja logica.
Calculable	Pedido implementa calcularTotal() y lo usa al agregar detalles
Relación 1:1	Usuario contiene un PerfilUsuario propio, unidireccional y no compartido.
Menú y errores	ConsoleReader valida enteros, decimales, confirmaciones y opciones numericas.
Reproducibilidad	README con requisitos, compilacion, ejecucion y lugar para links de video y PDF.

8. Bibliografia y webgrafia

- Documentacion oficial de Java: <https://docs.oracle.com/en/java/>
- Java Collections Framework: <https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/package-summary.html>
- Guia de lenguaje Java: <https://docs.oracle.com/javase/tutorial/java/>
- Material de la catedra: Consigna y pautas de correccion del TPI Programacion 2.

9. Links de entrega

Video demostrativo publico: <https://www.youtube.com/watch?v=39e43ybhUQs&themeRefresh=1>